

EventPass Project Documentation

22424830

August 13, 2026

Contents

1 Project Title	1
2 Problem Statement	1
3 Aim and Objectives	1
4 Stakeholders	1
5 Requirements Analysis	1
6 SRS Summary	2
7 Software Effort Estimation	2
8 System Analysis	2
9 System Design	2
10 Implementation	2
11 Testing	2
12 Technical Debt	2
13 Deployment	2
14 User Manual	2
15 Maintenance Strategy	3
16 Future Evolution	3
17 Limitations	3
18 Conclusion	3

1 Project Title

EventPass — Event Ticketing & Check-in System

2 Problem Statement

Event organizers need a lightweight system to manage attendees, assign ticket codes, and perform fast check-ins. EventPass addresses this with a focused web-based workflow under a 48-hour exam constraint.

3 Aim and Objectives

- Create and manage events
- Register attendees and generate unique tickets
- Provide QR-based ticket presentation
- Support fast check-in by staff
- Enforce organizer-only access to data

4 Stakeholders

- Event organizers
- Check-in staff
- Attendees
- Examiners

5 Requirements Analysis

This project prioritizes a minimal, correct core workflow. The system is intentionally scoped to a single organizer role, event registration, attendee management, and concurrency-safe check-ins.

6 SRS Summary

The system supports organizer registration and login, event creation, attendee registration, unique ticket codes, and ticket lookup/check-in. Non-functional requirements focus on security, fast check-in, and data isolation.

7 Software Effort Estimation

Selected technique: expert estimation. Estimated effort: 40-50 person-hours over 48 hours. Scope was controlled to fit the time constraint.

8 System Analysis

The architecture uses a React frontend, Express backend, and SQLite database. The main business flow is organizer → event → attendee → ticket → check-in.

9 System Design

The design separates presentation, API, and persistence layers. The most critical design decision is atomic SQL update logic for safe concurrent check-ins.

10 Implementation

Core features implemented: authentication, event management, attendee registration, ticket generation, QR output, and check-in processing.

11 Testing

Backend tests validate ticket generation and check-in behavior, especially concurrency. The frontend build was also verified successfully.

12 Technical Debt

The project carries deliberate debt such as SQLite-only design, simple auth model, and limited security hardening. This is documented in the technical debt plan.

13 Deployment

The project includes a Render deployment config and local deployment instructions for a hosted environment.

14 User Manual

Users register, log in, create events, add attendees, and process check-ins via ticket codes or QR scans.

15 Maintenance Strategy

Corrective, adaptive, perfective, and preventive maintenance are all relevant. Future work includes better role management, scale-up, and security hardening.

16 Future Evolution

Potential additions include payment integration, role-based access, email notifications, analytics, and migration to PostgreSQL.

17 Limitations

No attendee self-service portal; no payment system; no full admin role model; scaling beyond a small event is limited.

18 Conclusion

EventPass demonstrates a realistic capstone-style delivery that fits within the 48-hour exam scope while still covering the software engineering lifecycle.